TCM-AI

Ziffernerkennung mit Künstlichen Neuronalen Netzen
Auf MNIST-Basis

Präsentiert von:
Tim, Cédric und Mika

März 2026

1. Projektüberblick

Ziel dieses Projekts war die vollständige Eigenimplementierung eines künstlichen neuronalen Netzes zur Klassifikation handgeschriebener Ziffern (0–9). Dabei wurden keine Machine-Learning-Frameworks wie TensorFlow oder PyTorch verwendet. Stattdessen basiert die gesamte Implementierung auf NumPy beziehungsweise auf pyOpenCl für die GPU.

1.1 End-to-End-Datenfluss

1. Laden der Bilddaten aus Trainings- und Testordnern
2. Vorverarbeitung (Resize auf 28×28, Normalisierung auf [0,1], Flatten zu 784 Werten, im fall dass die Trainingsdaten Fehler hätten)
3. Forward-Pass durch das Multi-Layer-Perceptron
4. Loss-Berechnung (Cross-Entropy)
5. Backpropagation & Gewichtsupdate
6. Evaluation auf Testdaten
7. Speichern der Modellartefakte (.npz, .json, Trainingsplot)

1.2 Projektstruktur

nn.py – Implementierung des neuronalen Netzes
train.py – CPU-Training
train-gpu.py – Optionales GPU-Training via “pyOpenCL”
app.py – Zeichen-UI zur Modellverwendung
models/ – gespeicherte Modelle
data/ – Trainings- und Testdaten

2. Theoretische Grundlagen

Ein künstliches Neuron berechnet eine gewichtete Summe der Eingaben: $z = w \cdot x + b$. Anschliessend wird eine Aktivierungsfunktion angewendet: $a = f(z)$. Erst durch nichtlineare Aktivierungen kann das Netz komplexe Muster lernen.

Hidden Layers: ReLU (max(0, z))

Output Layer: Softmax (Wahrscheinlichkeitsverteilung über 10 Klassen)

Loss-Funktion: Cross-Entropy

3. Modellarchitektur

Input: 784 Neuronen (28×28 Pixel)
Hidden Layers: 1–3 Schichten (Profil mini/normal/pro)
Output: 10 Neuronen (Ziffern 0–9)

Die Gewichte werden zufällig initialisiert, Biases starten bei 0.

4. Training & Optimierung

Training erfolgt mit Mini-Batch Gradient Descent. Nach jedem Forward-Pass wird der Fehler berechnet und mittels Backpropagation durch das Netzwerk propagiert.

4.1 Backpropagation

1. Forward-Pass (Speicherung aller Aktivierungen)
2. Fehler am Output: (p − y)
3. Rückwärtspropagation der Gradienten
4. Update von Gewichten und Biases

4.2 Optimierer

Frühe Modelle nutzten Standard-SGD. Ab Version o4.1 wurde der Adam-Optimizer eingeführt. Adam kombiniert Momentum mit adaptiven Lernraten pro Parameter und führte zu deutlich schnellerer Konvergenz sowie höherer Endgenauigkeit.

5. Ergebnisse & Evaluation

Die Modellgenauigkeit stieg mit zunehmender Datenmenge und verbesserter Architektur deutlich an. Während frühe Modelle (o1/o2) ca. 91–94% Test-Accuracy erreichten, lagen spätere Modelle (o4/o5) bei bis zu 99%.

Wichtige Erkenntnis: Modellgrösse allein garantiert keine bessere Performance. Entscheidend sind Datenmenge, Architektur, Optimierer und Hyperparameter.

6. Learnings & Entwicklung über Generationen

Generation o1 → Erste funktionierende Version

- 1 Hidden Layer
- Interner 80/20 Split der 10’000 Trainingsdaten aus dem Reduced-MNIST Datenset
- SGD

Ergebnis: Solide Basis, aber begrenzte Generalisierung. Ca. 91% Accuracy bei den Tests

Generation o2 → Reproduzierbarkeit & bessere Parameter

- Fester Random Seed (42)
- Optimierte Lernrate

Ergebnis: +2–3% Accuracy, stabilere Resultate. (94%)

Generation o3 → Full Dataset

- Wechsel von Reduced MNIST (10k) zu Full MNIST (60k)

Ergebnis: Deutliche Verbesserung der Generalisierung, grosse Fortschritte in der App und steigerung auf 96% im Benchmark.

Generation o4 → Mehr Hidden Layers

- 2 Hidden Layers für Normal- und Mini-Modell und 3 für das Pro-Modell
- Bessere Feature-Extraktion

Ergebnis: Weitere Steigerung der Genauigkeit zu maximalen 98% und ein schon fast gutes Erlebnis in der App

Generation o5 → Adam + Mega-Dataset

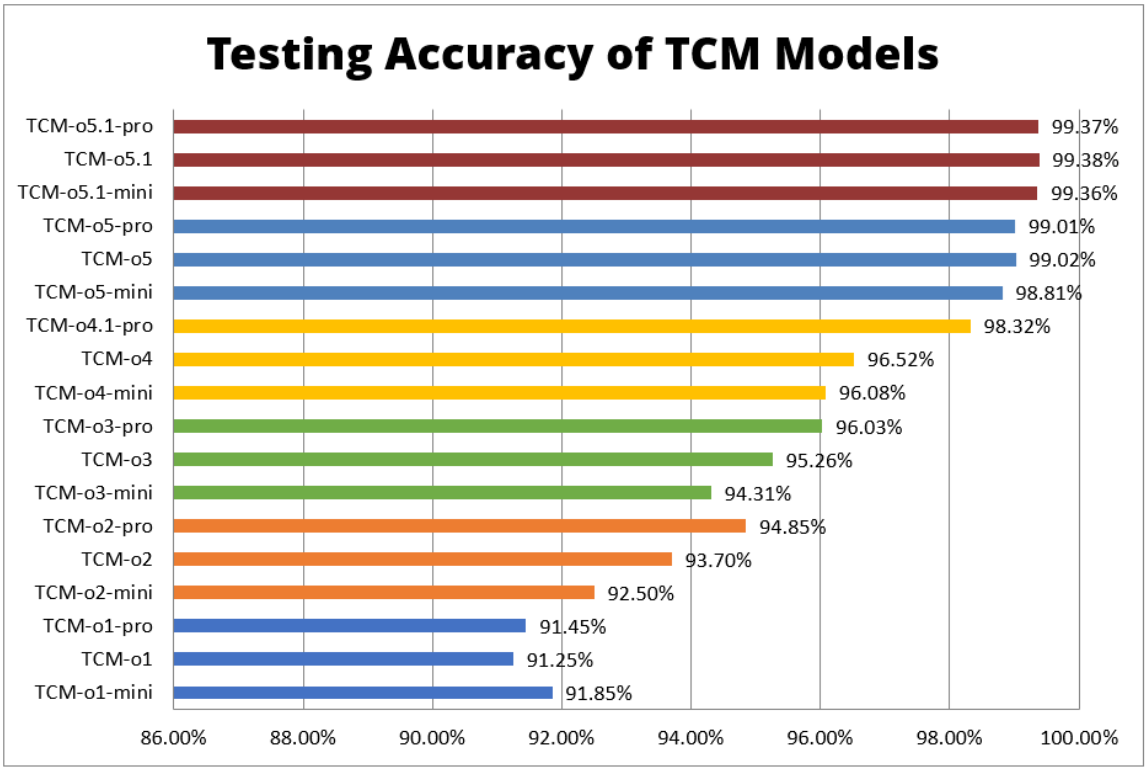
- Kombination MNIST + EMNIST (~300k Samples)
- Adam Optimizer
- Early Stopping

Ergebnis: ~99% Accuracy, sehr starke Generalisierung und ein Extrem gutes App-Erlebnis, aber das Grössere Pro-Modell mit 3 anstatt 2 Hidden layers und 4 mal mehr Neuronen schneidet gleich ab wie das Normal-Modell, ist aber in der App merkbar besser.

Generation o5.1 → Mega-Dataset + Data-Augmentation

- Data Augmentation beim Trainieren
- Restliche Parameter gleich wie o5-Familie

Ergebnis: Noch einmal bessere Accuracy und vermutlich Limit für diese Modellarchitektur.



7. Fazit

Das Projekt zeigte eindrucksvoll, wie sich neuronale Netze schrittweise verbessern lassen. Die grössten Performance-Sprünge entstanden durch:

1. Grössere und vielfältigere Datensätze

2. Mehrere Hidden Layers
3. Einsatz des Adam-Optimizers (Sprung von o4 zu o4.1-pro)

Besonders wichtig war die Visualisierung von Loss und Accuracy zur Analyse von Overfitting. Das Projekt ermöglichte ein tiefes Verständnis der mathematischen und praktischen Grundlagen künstlicher neuronaler Netze.